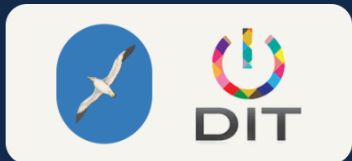


INGENIERÍA DE SOFTWARE III · UNIDAD 3

Puntos de Función

Medir funcionalidad para estimar esfuerzo



Facultad de Ingeniería · Sede Trelew · UNPSJB
Prof. Sebastián Schanz

C1 · LA PREGUNTA IMPOSIBLE

¿Cuántas líneas de código va a tener su sistema?

Nadie lo sabe. Y sin embargo hay que dar un plazo y un precio **antes** de escribir la primera línea.

La pregunta que sí se puede contestar

¿Cuántas pantallas
de carga tiene?

¿Cuántos reportes?

¿Cuántas tablas?

Eso se puede contestar **hoy**, con el enunciado en la mano, sin haber programado nada.



Ahí está toda la idea

Si no puedo medir el código, mido **lo que el sistema hace para el usuario**, y de ahí derivo cuánto trabajo cuesta construirlo.

SE PUEDE CONTAR

Atributos internos: pantallas, reportes, entidades.



SE DERIVA

Un atributo externo que todavía no se ve: el esfuerzo.

Es la misma jugada del puente: medir atributos internos que sí puedo contar, para derivar uno externo que no.

Puntos de Función

ALLAN ALBRECHT · IBM · 1979

Una medida del tamaño funcional del software.

Se cuenta lo que el sistema le ofrece al usuario, no cómo está construido por dentro.



Por qué es tan útil

En análisis

Se puede calcular en la etapa de análisis, con los requerimientos nada más.

Independiente

No depende del lenguaje ni de la tecnología.

Comparable

Permite comparar proyectos hechos con herramientas distintas.

Las líneas de código no cumplen ninguna de las tres.



El pedido del club

«Tenemos una liga de fútbol amateur en Trelew y manejamos todo en papel. Necesitamos cargar los equipos y los partidos: quién jugó contra quién, la fecha y el resultado. Los datos de los equipos cambian seguido — cambia el delegado, cambia la cancha donde hacen de local — así que hay que poder corregirlos, y cuando un equipo se baja, sacarlo.

La gente llama para saber cuándo juega tal equipo, así que hay que poder buscar un partido por fecha. Y para las reuniones necesitamos el listado de equipos, un reporte de los equipos anotados en un rango de fechas, y un reporte de los partidos jugados.»



Lo que pidieron, en limpio

Funciones

- Registrar equipos
- Registrar partidos
- Actualizar datos de equipos
- Eliminar equipos
- Buscar partido por fecha
- Listado de equipos
- Reporte de equipos por rango de fechas
- Reporte de partidos

Entidades

- Equipo
- Partido
- **Delegado**
- **Cancha**

C1 · EN DUPLAS · 5 MINUTOS

¿Cuántos tipos de cosas hay acá?

Esas funciones no son todas del mismo tipo. Algunas cargan datos, otras muestran, otras buscan. Y las entidades son otra cosa distinta.

¿En cuántos grupos las separarían?

El sistema se parte en dos

Todo sistema hace dos cosas: **procesa** y **guarda**.

PROCESA

Funciones de transacción

Lo que el usuario hace: entra datos, pide informes, consulta.

GUARDA

Funciones de datos

Lo que el sistema guarda, o consulta de otros sistemas.



Las tres funciones de transacción

EI

External Input

El usuario **mete** datos que cambian algo: altas, bajas, modificaciones.

EO

External Output

El sistema **entrega información elaborada:** informes, listados con totales, gráficos.

EQ

External Inquiry

El usuario **pide y ve** datos, sin cálculo y sin modificar nada.

La diferencia entre EO y EQ

Es la que más se confunde.

EQ

Recupera y muestra.

«*Buscar el partido del 12 de marzo.*»

EO

Recupera **y además procesa**: totaliza, calcula, ordena por un criterio derivado.

«*Reporte de goles por equipo del último semestre.*»

Las dos funciones de datos

ILF

Internal Logical File

Datos que **el sistema mantiene**. En la práctica, casi siempre una entidad del modelo de dominio.

EIF

External Interface File

Datos que el sistema **usa pero no mantiene**: los administra otro sistema.



«Archivo lógico» no es un archivo

ILF y EIF se cuentan desde el **punto de vista lógico**, no del sistema de archivos ni de la base de datos.

Una entidad con sus tablas auxiliares puede ser **un solo** ILF.

Un archivo físico puede no ser **ninguno**.

Ojo con la bibliografía

En los libros en castellano vas a encontrar las siglas traducidas:

Inglés (IFPUG)	Castellano
EI · External Input	EE — Entrada Externa
EO · External Output	SE — Salida Externa
EQ · External Inquiry	CE — Consulta Externa
ILF · Internal Logical File	ALI — Archivo Lógico Interno
EIF · External Interface File	AIE — Archivo de Interfaz Externo

Son las mismas cosas. Acá usamos las inglesas, que son las del estándar IFPUG.

Cómo se reconocen en la práctica

Lo que dice el requerimiento	Suele ser
Insertar, Registrar, Actualizar, Eliminar	EI
Listar, Informe, Reporte	EO
Buscar, Consultar, Ver	EQ
Entidad que el sistema mantiene	ILF
Datos que administra otro sistema	EIF

Pero no es automático

La tabla anterior es una **guía, no una regla**. No todo informe es EO ni toda tabla de la base es un ILF.

Hay que mirar qué hace la función para el usuario, no cómo se llama.



No todo pesa igual

Cada componente se pondera según su complejidad. La tabla se fija antes de empezar:

Componente	Simple	Medio	Complejo
EI · External Input	3	4	6
EO · External Output	4	5	7
EQ · External Inquiry	3	4	6
ILF · Internal Logical File	7	10	15
EIF · External Interface File	5	7	10

C2 · PESOS Y CONTEO · EN DUPLAS

Ahora sí, a contar

Volvemos al sistema de fútbol del comienzo. Ya sabemos cómo se llama cada tipo de función y cuánto pesa cada una.

Clasifiquen cada función, y después pesen.

El conteo

Función	Tipo	Cuenta × Peso	PF
Registrar equipos	EI	1 × 4	4
Registrar partidos	EI	1 × 4	4
Actualizar datos de equipos	EI	1 × 4	4
Eliminar equipos	EI	1 × 4	4
Buscar partido por fecha	EQ	1 × 4	4
Listado de equipos	EO	1 × 5	5
Reporte de equipos por fechas	EO	1 × 5	5
Reporte de partidos	EO	1 × 5	5
Entidades del sistema	ILF	4 × 10	40

Todas las funciones se tomaron de complejidad media.

PUNTOS DE FUNCIÓN SIN AJUSTAR · SISTEMA DE FÚTBOL

PFSA = 75

40 de los ILF · 4 entidades × 10 **35** de las 8 funciones de transacción

Ya tenemos una medida del tamaño del sistema, y todavía no escribimos una línea de código.

Ahora les toca

Cuenten los puntos de función sin ajustar de un sistema chico, con el enunciado a la vista.



En ese orden.

Dos sistemas, los mismos PF

SISTEMA A · 75 PF

Corre en una notebook, para diez usuarios internos.

SISTEMA B · 75 PF

Distribuido, en seis sedes, con datos sensibles y respuesta en tiempo real.

¿Cuestan lo mismo?



Los factores de ajuste

Albrecht listó **14 características** que hacen que un sistema cueste más o menos con la misma funcionalidad.

NO SON SOBRE

qué hace el sistema

SINO SOBRE

**en qué condiciones tiene
que hacerlo**



Los 14 factores

#	Factor	#	Factor
1	Comunicación de datos	8	Actualización en línea
2	Procesamiento distribuido	9	Procesamiento complejo
3	Rendimiento	10	Reusabilidad
4	Uso intensivo de la configuración	11	Facilidad de instalación
5	Tasa de transacciones	12	Facilidad de operación
6	Entrada de datos en línea	13	Múltiples sitios
7	Eficiencia para el usuario final	14	Facilidad de cambio

Cómo se puntúa cada factor

De **0 a 5**, según cuánta influencia tiene en este sistema:

0 Ninguna

1 Incidental

2 Moderada

3 Media

4 Significativa

5 Esencial

Suma total de los 14 factores (ΣFi): entre **0 y 70**.

La fórmula del ajuste

$$\text{PFA} = \text{PFSA} \times [0,65 + (0,01 \times \Sigma \text{Fi})]$$

Término	Qué es
PFSA	Puntos de función sin ajustar
ΣFi	Suma de los 14 factores (0 a 70)
PFA	Puntos de función ajustados

El ajuste del sistema de fútbol

Con PFSA = 75 y $\Sigma Fi = 32$:

$$PFA = 75 \times [0,65 + (0,01 \times 32)]$$

$$PFA = 75 \times [0,65 + 0,32]$$

$$PFA = 75 \times 0,97 = 72,75$$

RESULTADO

PFA $\approx$ 73

El ajuste mueve poco

Con $\Sigma Fi = 0$

0,65

-35%

Con $\Sigma Fi = 35$

1,00

sin cambio

Con $\Sigma Fi = 70$

1,35

+35%

El ajuste solo puede mover el resultado $\pm 35\%$. Lo que de verdad define el tamaño es **el conteo**, no los factores.

Del tamaño al trabajo

Los PF miden **cuán grande** es el sistema. Todavía no dicen cuántos meses ni cuánta plata.

Para eso hay que convertir, y ahí entran los **datos históricos** de la organización.



De PF a líneas de código

Cada lenguaje necesita más o menos código para la misma funcionalidad:

Lenguaje	LDC por PF (aprox.)
Assembler	~320
C	~128
Java / C#	~55
JavaScript / TypeScript	~47
Python	~30
SQL	~15
Generadores de código	~15

Un sistema, tres lenguajes

La ISFPP se hace en **React + FastAPI + SQLite**. No hay un único número de LDC por PF que sirva para todo el sistema.

FRONTEND

React

BACKEND

FastAPI

BASE DE DATOS

SQLite

Hay que **repartir la funcionalidad por capa** y convertir cada parte con su propio factor.



Cómo se reparte

Estimar qué porcentaje de los PF corresponde a cada capa, y convertir por separado:

Capa	Lenguaje	Reparto estimado	Factor
Frontend	React / TypeScript	~45%	47
Backend	Python / FastAPI	~45%	30
Base de datos	SQL	~10%	15

Se suman las tres. **El reparto se justifica, no se inventa**, y se corrige al final con las LDC reales.

De PF a esfuerzo

La vía corta: **productividad**, medida en PF por persona-mes.

SI

el equipo produce **6,5 PF por persona-mes** y el sistema tiene **372 PF**:

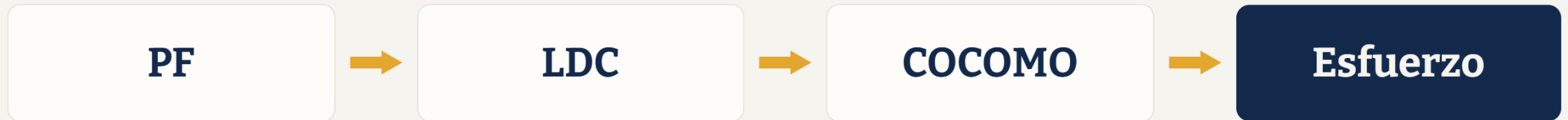
$$372 \div 6,5$$

≈ 57 personas-mes



La otra vía

También se puede pasar por las líneas de código:



Sirve cuando hay datos históricos de LDC pero no de PF. Da parecido, no igual.

De esfuerzo a plazo

El esfuerzo no se divide por la gente disponible. 57 personas-mes no son 57 personas en un mes.

Sumar gente a un proyecto atrasado lo atrasa más: hay que capacitarlos y aumentan los canales de comunicación.



De esfuerzo a costo

$$\text{Costo} = \text{esfuerzo (personas-mes)} \times \text{costo mensual cargado de una persona}$$

«Cargado» quiere decir el costo real:

Sueldo

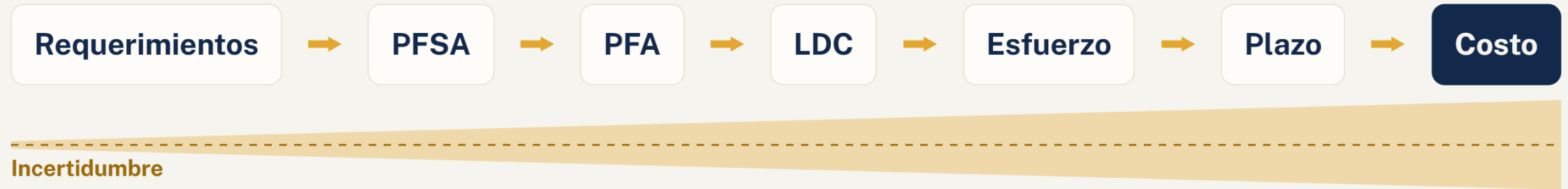
Cargas sociales

Infraestructura

Licencias

La parte proporcional de lo que no factura

La cadena completa



Cada flecha suma incertidumbre. Al final no hay que dar un número: hay que dar un **rango**, y decir de dónde salió.

Un número no es un compromiso

Estimar

Calcular cuánto podría costar.

Comprometerse

Prometer una fecha.

Son dos actos distintos y los decide gente distinta. Confundirlos es el origen de la mitad de los proyectos que fracasan.

Lo que PF no hace

No mide calidad

Un sistema de 300 PF puede estar bien o mal hecho.

No mide dificultad algorítmica

Un motor de cálculo pesa poco en PF.

No compara personas

No sirve para comparar personas. Nunca.



Las críticas honestas

Cuesta trabajo contarlos

Y los proyectos suelen ir con el tiempo justo.

Depende de quién cuenta

Hay que fijar criterios y sostenerlos.

Pierde precisión

En sistemas de mucho cálculo y poca interacción.

Aun así, se puede calcular en análisis. Ninguna otra medida de tamaño puede.



Equivocarse es parte del método

Ninguna estimación es exacta. Lo que la vuelve útil es **quedar escrita** y después **compararse contra lo que pasó**.

ESTIMA Y NUNCA REVISA

Una organización así no mejora nunca.

REGISTRA SUS ERRORES

Calibra sus números, y en dos o tres proyectos estima bastante bien.



EL HILO EN UNA FRASE

No podemos medir el código que todavía no escribimos. Pero sí podemos contar lo que el sistema va a hacer, y con eso, y con lo que nos pasó antes, poner un número que se pueda defender.

UNIDAD 3 · CIERRE

Estimar en ágil

Story points, planning poker y velocity

C8 · LA PREGUNTA INCÓMODA

Acabamos de estimar un sistema entero con Puntos de Función. Los que estén trabajando:

¿Alguno usa Puntos de Función en su trabajo?

No es que PF esté mal

PF y las técnicas ágiles resuelven **momentos distintos** del mismo problema.

Cuál sirve cuándo, y por qué en realidad son de la misma familia de ideas.



Story points

En Scrum, las historias de usuario no se estiman en horas: se estiman en **puntos**.

Un punto no es una unidad de tiempo. Es una medida de **tamaño relativo** que combina:

Volumen de trabajo

Complejidad

Incertidumbre

Por qué relativo y no absoluto

ABSOLUTO

«¿Cuántos kilos pesa ese mueble?»

Difícil, y la mayoría erra feo.

RELATIVO

«¿Pesa más que aquel otro?»

Fácil, y casi nadie se equivoca.

Somos **malos estimando en absoluto y buenos comparando**. Los story points aprovechan eso: se elige una historia de referencia y todo se mide contra ella.



Planning poker

Cómo se estima una historia en equipo:

1

Se lee la historia y se aclara lo que haga falta.

2

Cada uno elige su carta en privado, sin decir nada.

3

Se revelan todas a la vez.

4

Los extremos, el más alto y el más bajo, explican por qué.

5

Se vuelve a votar hasta converger.

ESTO YA LO VIMOS

Estimación individual y en secreto

Revelación simultánea

Discusión de las diferencias

Nueva ronda

Es Delphi banda ancha.

Planning poker es su versión con cartas, de 2002.

La misma idea, tres veces

U2

El antídoto contra el **anclaje**:
que cada uno escriba
su número sin ver el de
los demás.



U3

Método formal de
estimación: **Delphi**
banda ancha.



INDUSTRIA

Se llama **planning poker** y
se juega con cartas.

Es la misma técnica. Lo que resuelve siempre es lo mismo: **que el primero que habla no arrastre al resto.**

Velocity

Los puntos que el equipo **efectivamente completa** en un sprint.

Después de dos o tres sprints hay un promedio, y con eso se puede anticipar cuántos puntos entran en el próximo.



Velocity es una predicción

Estimar

Medir el largo del puente y el de un auto para derivar cuántos entran. Se miden atributos internos.

Puntos de Función

Predecir

Contar los autos de los sábados anteriores y proyectar. Se usa la historia.

Velocity



Por eso sirven en momentos distintos

	Puntos de Función	Velocity
Cuándo	Antes de empezar	Después de 2-3 sprints
Necesita	Los requerimientos	Historia del equipo
Da	Un tamaño absoluto	Un ritmo del equipo
Sirve para	Cotizar, contratar, comparar	Planificar el próximo sprint

Sin historia no hay velocity. **Y el primer día de un proyecto nunca hay historia.**

La trampa de velocity

Los puntos de un equipo **no son comparables con los de otro**: cada equipo calibra su propia escala contra su propia historia.

Usar velocity para comparar equipos, o para exigir que suba, es el caso visto en métricas:

la métrica se convierte en objetivo y los puntos se inflan.



Lo que comparten

Ninguna de las dos funciona sacada de una caja.

Puntos de Función

Necesita **tus** tablas de conversión y **tu** productividad.

Velocity

Necesita **tus** sprints anteriores.

Las dos son inútiles sin datos propios, y esos datos salen de **medir**.



EL HILO DE LA UNIDAD, EN UNA FRASE

Se mide para poder estimar. Se estima para poder decidir. Y se guarda lo que pasó, porque la próxima estimación se hace con eso.